

NightShift — Execution Plan (V7.4)

Google & Kaggle 5-Day AI Agents Intensive (Vibe Coding) — Capstone Track: Agents for Business

NightShift Team (Members A, B, C, D)

July 2026

Table of Contents

1 NightShift — Execution Plan (V7.4)

- 1.1 0. Capstone & Course Alignment
 - 1.1.1 0.1 Official capstone submission (all mandatory)
 - 1.1.2 0.2 Scoring rubric (100 points) — plan coverage
 - 1.1.3 0.3 Course curriculum → NightShift build mapping
 - 1.1.4 0.4 Key Concepts checklist (need 3 of 6 — target 5 of 6)
 - 1.1.5 0.5 Technical Implementation (50 pts) — TDD mandatory features
 - 1.1.6 0.6 Compliance gaps fixed in this revision
 - 1.1.7 0.7 Non-negotiable tech stack (TDD §2.1)
 - 1.1.8 0.8 Kaggle daily discussion threads → unit requirements
 - 1.1.9 0.9 Course Day 4 — Effective Trust: 7 pillars → NightShift mapping
 - 1.1.10 0.10 Gaps addressed after course thread review (this revision)
 - 1.1.11 0.11 V7.4 revision — operational playbook merge
- 1.2 1. Executive Summary
 - 1.2.1 The three things that will actually win or lose this
 - 1.2.2 Vertical-slice philosophy
- 1.3 2. Team Roles & Ownership
 - 1.3.1 2.1 Assumptions & constraints
 - 1.3.2 2.2 Tools & access matrix
- 1.4 3. Shared Contracts (do not change without team sync)
 - 1.4.1 3.1 Pydantic models (`models/core.py`)
 - 1.4.2 3.2 MCP tool contracts (Member A owns implementation)
 - 1.4.3 3.3 HITL state machine (Member C owns enforcement)
 - 1.4.4 3.4 Directory scaffold (Member A creates on Day 1)
 - 1.4.5 3.5 Environment & dependencies (Member A — Day 1)
 - 1.4.6 3.6 Memory store contracts (Member B — Day 1)
 - 1.4.7 3.7 MCP fixture & eval shape (Member A + B — Day 1)
 - 1.4.8 3.8 Agent Skills template (course Day 3 — progressive disclosure)
 - 1.4.9 3.9 Repo governance files (course Day 5)
 - 1.4.10 3.10 Environment onboarding (all members — Day 1 AM)
- 1.5 4. Day-by-Day Schedule
 - 1.5.1 4.0 Milestones at a glance (M1-M10)
 - 1.5.2 Schedule at a glance (Gantt summary)
 - 1.5.3 Day 1 — Wednesday, July 1: Foundation & Plumbing
 - 1.5.4 Day 2 — Thursday, July 2: Ingestion + Triage Live
 - 1.5.5 Day 3 — Friday, July 3: Integration + First End-to-End Run

- 1.5.6 Day 4 — Saturday, July 4: Quality, Security & Hardening
- 1.5.7 Day 5 — Sunday, July 5: Demo Polish & Submission Assets
- 1.5.8 Day 6 — Monday, July 6: Submit
- 1.5.9 Day 7 — Tuesday, July 7: Buffer only (optional)
- 1.6 5. Member Workstreams (Detailed)
 - 1.6.1 5.1 Team Member A — Platform, Supervisor & MCP Ingestion
 - 1.6.2 5.2 Team Member B — Triage, Memory & Evaluation
 - 1.6.3 5.3 Team Member C — HITL Safety, Drafting & Morning Brief
 - 1.6.4 5.4 Team Member D — Observability, Security & Submission
- 1.7 6. Dependency Graph & Handoff Points
 - 1.7.1 Critical handoffs (blocking)
 - 1.7.2 Communication rhythm
 - 1.7.3 6.1 Git & code review policy
 - 1.7.4 6.2 Escalation path
 - 1.7.5 6.3 Shared resources
- 1.8 7. Rubric & Requirements Traceability
 - 1.8.1 7.1 Kaggle rubric → owner
 - 1.8.2 7.2 Key Concepts checklist (need 3 of 6; target 5 of 6)
 - 1.8.3 7.3 TDD build priority → day mapping
 - 1.8.4 7.4 PRD success metrics → owner
 - 1.8.5 7.5 Course Day 5 (SDD + observability) → deliverable checklist
- 1.9 8. Final Submission Checklist
 - 1.9.1 8.1 Code & repo
 - 1.9.2 8.2 Kaggle portal (four mandatory assets)
 - 1.9.3 8.3 Writeup content checklist (PRD §3.1)
 - 1.9.4 8.4 Video checklist (PRD §3.2)
 - 1.9.5 8.5 Security final pass
- 1.10 9. Risk Register & Mitigations
- 1.11 10. Testing & Validation Playbook
 - 1.11.1 10.1 Continuous eval (daily from Day 2)
 - 1.11.2 10.2 Hard-case validation (Day 3 — blocking)
 - 1.11.3 10.3 State machine validation (Day 1 — screenshot for Writeup)
 - 1.11.4 10.4 DB integrity query (Day 5 — screenshot for Writeup)
 - 1.11.5 10.5 Git hygiene scan (Day 5 — Member A)
 - 1.11.6 10.6 Reproducibility test (Day 5 — Member A)
 - 1.11.7 10.7 ResponseAgent draft quality (Day 5 — Member C)
- 1.12 11. Stretch Goals (only after Day 4 checkpoint green)
- 1.13 12. PDF vs This Plan — Gap Summary
- 1.14 13. Claude V7.2 Markdown vs V7.4 — Gap Summary
- 1.15 14. Copilot V1.0 vs This Plan — Gap Summary
- 1.16 15. Reference Implementation Patterns (starter code)
 - 1.16.1 15.1 ADK supervisor skeleton (`main.py` — target shape)
 - 1.16.2 15.2 IngestionAgent MCP client pattern (inside ADK IngestionAgent)

- 1.16.3 15.3 Eval confusion matrix (`tests/eval_urgency.py`)
- 1.16.4 15.4 Architecture diagrams for README
- 1.17 16. Quick Reference Commands (target state)

1 NightShift — Execution Plan (V7.4)

Google & Kaggle 5-Day AI Agents Intensive (Vibe Coding) — Capstone Track: Agents for Business

Companion documents: [NightShift_PRD-V7.3.md](#) (what/why) · [NightShift_TDD-V7.3.md](#) (how)
Related: [NightShift – Team Execution Plan-Claude.pdf](#) · [NightShift_ExecutionPlan-claudeV7.2.md](#) (operational team playbook — merged in V7.4 §2.1, §4.0, §6.1–6.3, §10) · [NightShift – Detailed Execution Plan-V1.0-copilot-.md](#) (starter code & repo layout — merged in §3.5, §15)
Team: **Member A**, **Member B**, **Member C**, **Member D**
Build window: 6 days (Jul 1–6) · **Submission deadline:** July 6, 2026, 11:59 PM PT
Submit target: July 6, 2026, **6:00 PM PT** (6-hour buffer before hard deadline — Kaggle platform issues, no grace period, single submission only)

Capstone alignment: This plan is validated against [NightShift_PRD-V7.3.md](#) §0–§3.5, [NightShift_TDD-V7.3.md](#) §2.1–§2.8, the [Kaggle Learn Guide](#), and the official daily discussion threads (linked in §0.8).

1.1 0. Capstone & Course Alignment

This section confirms the execution plan satisfies the **Google & Kaggle 5-Day AI Agents Intensive** capstone requirements before any code is generated.

1.1.1 0.1 Official capstone submission (all mandatory)

Asset	Requirement	Owner	Plan section
Kaggle Writeup	≤2,500 words; Submitted (drafts do not count)	D	§5.4, §8.3
Media Gallery	Cover image required	D	§8.2
YouTube video	Public , ≤5 minutes	D	§5.4, §8.4
Public Project Link	Public GitHub repo with working setup instructions (live demo optional)	A, D	§8.1

Not required for judging: cloud deployment, billing account, live public endpoint (TDD §2.6; optional polish only).

Eligibility: team of up to 4 (this plan uses 4) · **one track** per submission · **one final submission** (no post-deadline iteration) · submit early (target **Jul 6, 6:00 PM PT**).

Recognition: every **qualifying capstone submission** earns a Kaggle badge and certificate. **Top 3 teams per track** receive swag and social recognition (winning is optional; badge requires valid submission).

Track (final): Agents for Business — property-management workflow automation (PRD §0.5.4). Kaggle may reassign a submission to a different track during judging — state your choice once and build a coherent Business narrative. Alternatives: Agents for Good, Concierge Agents, Freestyle.

1.1.2 0.2 Scoring rubric (100 points) — plan coverage

Category	Pts	What judges evaluate	Where this plan delivers
Core Concept & Value	10	Problem, business value, why agents	Headline hook §1; Writeup §8.3; video hook §5.4
YouTube Video	10	≤5 min; problem → why agents → architecture → demo → build	§5.4 five beats; hard-case demo ~2 min
Writeup	10	Architecture, journey, key concepts	§8.3; TDD §2.7 decision log
Technical Implementation	50	ADK multi-agent graph, MCP, memory, sessions, security, SDD, observability	§0.3, §0.4, §5.1–5.3, §7.6
Documentation (README)	20	Problem, solution, architecture, setup, diagrams	§5.4 D5; §8.1

1.1.3 0.3 Course curriculum → NightShift build mapping

The course teaches concepts over 5 **learning days**; this plan implements them across **6 calendar build days** (Jul 1–6):

Course day	Course topic	NightShift implementation	Owner	Calendar
Day 1	Agent foundations, ADK intro	ADK 2.0 supervisor graph + 3 stub agents; <code>models/core.py</code> ; repo scaffold	A	Jul 1
Day 2	Tools, MCP, multi-agent orchestration	MCP server (<code>read_inbox</code> , <code>read_hoa_portal</code> , <code>read_invoices_folder</code>); <code>IngestionAgent</code> ; code-execution invoice audit	A	Jul 1–2
Day 3	Sessions, context engineering, memory, skills	Per-item classification (no mega-prompt); context pruning <code>RawItem→ClassifiedItem</code> ; memory lookup; <code>SKILL.md</code> per agent; ADK session per overnight run	A, B, C	Jul 2–3
Day 4	Security, guardrails, evaluation	Red/Blue/Green triad; PII scoping; <code>eval_urgency.py</code> + confusion matrix; threat model in Writeup	B, D	Jul 3–4
Day 5	Spec-driven development, observability, deploy	Gherkin specs as source of truth; policy check script; OpenTelemetry traces; optional Docker/Cloud Run	D, A	Jul 4–5
Capstone	Integrate + submit	README, video, Writeup, four assets, regression	All	Jul 5–6

1.1.4 0.4 Key Concepts checklist (need 3 of 6 — target 5 of 6)

Per PRD §0 — each must be **demonstrable**, not just mentioned:

#	Key Concept	Required evidence	Owner	Day
1	Agent / Multi-agent (ADK 2.0)	Three distinct ADK sub-agents + <code>SupervisorNode</code> in <code>main.py</code> — not a single monolithic prompt	A	1
2	MCP Server	Read-only tools with bearer auth; mock fixtures	A	1–2
3	Security features	HITL DB constraint + Red/Blue/Green guardrails	C, D	1, 4
4	Agent skills / Agents CLI	<code>SKILL.md</code> per agent; runnable via <code>agents-cli</code> or documented CLI	B, C, A	2–3
5	Deployability	Dockerfile + README scaling notes (optional live Cloud Run)	A, D	4–5
6	Antigravity	Mention in video: built in Cursor as equivalent agentic IDE	D	5

1.1.5 0.5 Technical Implementation (50 pts) — TDD mandatory features

These are scored under Technical Implementation (not the 6-item Key Concepts list) but are **required by the TDD** and course:

TDD feature	Requirement	Owner	Deliverable	Day
Multi-agent graph	ADK 2.0; 3 agents; supervisor routing	A	<code>main.py</code> , <code>agents/*/</code>	1
MCP tools	3 read-only tools; <code>RawItem</code> contract	A	<code>mcp/server.py</code>	1–2
ADK session	One session per overnight batch; shared run context; reconstructable state	A	<code>session/</code> or ADK session config + SQLite run state	2
Context engineering	Per-item calls; prune <code>raw_text</code> ; programmatic Morning Brief	B, C	<code>TriageAgent</code> , <code>ResponseAgent</code>	2–3
Long-term memory	Tenant→property lookup; personality notes; no mid-run memory writes	B	<code>memory/store.py</code> ; nightly consolidation script	2–3
Code execution tool	Invoice line-item audit (sandbox fallback OK)	A	Triage/ingestion tool module	4
Security / Effective Trust	Red/Blue/Green; PII redaction; threat model in Writeup	D	<code>security/</code>	4
Evaluation loop	20+ fixtures; confusion matrix; hard case	B	<code>tests/eval_urgency.py</code>	2–4
Spec-driven development	Gherkin specs + policy check (no <code>sent</code> bypass)	D	<code>features/*.feature</code> , <code>policy/check_no_send.py</code>	1, 4
Observability	OTel: model path, tokens, tool latency	D	<code>observability/tracing.py</code>	2–4
HITL state machine	DB <code>CHECK</code> constraint + FSM validator	C	<code>db/models.py</code> , <code>db/fsm.py</code>	1
A2A protocol	Intentionally not used (document in Writeup per TDD §2.7)	D	Writeup §8.3	5

1.1.6 0.6 Compliance gaps fixed in this revision

Gap	Risk	Fix in this plan
Reference code used plain Python only	Fails ADK Key Concept	§15 now requires ADK graph; ThreadPoolExecutor is scheduling inside supervisor only
ADK session not explicit	Misses Day 3 course standard	§0.5, deliverable A2b
<code>SKILL.md</code> not in deliverables	Misses Agent skills concept	Deliverables A4b, B2b, C3b
Policy Server only in stretch goals	Misses Day 5 SDD standard	Required lightweight <code>policy/check_no_send.py</code> Day 4
Memory mid-run writes	Violates TDD §2.3	Nightly consolidation deliverable B1b
<code>agents-cli</code> not in commands	Misses Key Concept #4	§16 commands + README requirement
<code>google-adk</code> not in dependencies	Stack mismatch with TDD §2.1	§3.5 dependencies

1.1.7 0.7 Non-negotiable tech stack (TDD §2.1)

- **Orchestration:** Python + Google ADK 2.0 (stateful code-first multi-agent graph)
- **LLM:** Gemini 1.5 **Flash** (classification) + **Pro** (drafting) via Google AI Studio API
- **Local run:** `agents-cli` or documented equivalent CLI wrapping `main.py`
- **IDE:** Cursor (Antigravity equivalent — state in video)
- **Secrets:** `.env` locally; never in repo or traces
- **Optional deploy:** Docker → Cloud Run (no billing required for submission)

1.1.8 0.8 Kaggle daily discussion threads → unit requirements

Source note: Kaggle discussion pages are JavaScript-rendered and could not be scraped directly. Requirements below are cross-checked against the [official Learn Guide](#), Day 5 wrap-up livestream (discussion [709712](#)), and course whitepapers/podcasts linked from each thread.

Day	Discussion thread	Official unit	NightShift must demonstrate
1	708280	Introduction to Agents & Vibe Coding	Factory model: developer as orchestrator of Context + Constraint + Evaluation harnesses (not syntax writer). NightShift maps: Context = memory/MCP; Constraint = HITL DB + policy; Evaluation = <code>eval_urgency.py</code> + Gherkin
2	708469	Agent Tools & Interoperability	MCP server with clear tool descriptions; granular read-only tools; bearer auth; meaningful error responses. A2A/A2UI/AP2 taught but not required — document A2A scope in Writeup
3	708744	Agent Skills	<code>SKILL.md</code> per agent with YAML frontmatter (<code>name</code> , <code>description</code>); progressive disclosure — skills loaded on demand, not stuffed into one mega-prompt; Agents CLI run/test
4	709165	Security & Evaluation	Effective Trust 7-pillar mapping (§0.9); expense-approval HITL is the course reference pattern — NightShift is the same shape with stricter HITL (no auto-send even for GREEN)
5	709464	Spec-Driven Development	Gherkin = source of truth; code disposable; <code>AGENTS.md</code> repo rules; hybrid policy check; OTel trajectories. Optional codelabs (Agent Runtime, Pub/Sub UI) — stretch only
—	709712	Wrap-up & capstone	Four tracks; submit early; badge for all qualifying submissions; incorporate course concepts in final project

1.1.9 0.9 Course Day 4 — Effective Trust: 7 pillars → NightShift mapping

Judges familiar with the course whitepaper expect you to name how your build addresses **Effective Trust**, not just “we added security.”

Pillar	Course definition	NightShift implementation	Owner
1	Ephemeral sandboxing	Invoice-audit code-execution tool (restricted function fallback OK per TDD §2.8)	A
2	Human-in-the-Loop	Headline feature: DB-enforced <code>staged → approved</code> ; manager Approve/Reject/Snooze; stricter than Day 4 expense agent (no auto-approve)	C
3	Indirect prompt injection defense	Red-team scan on inbound <code>raw_text</code> before classification	D
4	Credential scoping	MCP bearer token; per-property memory scope; no keys in traces	A, D
5	Slopsquatting protection	Review <code>requirements.txt</code> / <code>pyproject.toml</code> before commit; pin versions	D
6	Trajectory auditing	OpenTelemetry per-item traces (model, tokens, tool latency)	D
7	Dynamic safety guardrails	Blue/Green output validation; <code>policy/check_no_send.py</code> ; FSM validator	D, C

Red / Blue / Green triad (course Day 4): Red = inbound threat scan · Blue = sandbox + scoping · Green = eval harness + OTel + confusion matrix.

Writeup sentence to include: “NightShift extends the course’s Day 4 expense-approval HITL pattern to property management: every draft stays `staged` until explicit manager approval — including high-confidence GREEN items — with the guarantee enforced at the database layer.”

1.1.10 0.10 Gaps addressed after course thread review (this revision)

Gap found	Course source	Fix
Factory model (3 harnesses) not named	Day 1 whitepaper	§0.8 Day 1 row + Writeup §8.3 bullet
MCP tool descriptions / error messages	Day 2 whitepaper	§3.2 MCP notes; A3 deliverable
A2A taught but unused — no judge-facing explanation	Day 2	Writeup §8.3 + §0.5 A2A row
<code>SKILL.md</code> YAML frontmatter + progressive disclosure	Day 3 whitepaper	§3.11 template; B2/C3 deliverables
7-pillar Effective Trust not mapped	Day 4 whitepaper	§0.9 table
Expense-agent HITL parallel not cited	Day 4 codelab	§0.9 Writeup sentence; video “why agents” beat
<code>AGENTS.md</code> cross-tool foundation	Day 5 whitepaper	<code>AGENTS.md</code> at repo root (D, Day 1)
Pre-commit policy + security gate	Day 5 SDD	<code>.pre-commit-config.yaml</code> (Day 1 optional) + <code>scripts/pre_submit.sh</code> (Day 5)
Confused deputy / MCP read-only rationale	Day 2 security	Threat model in Writeup §8.3
<code>agents-cli</code> dry-run before deploy	Day 5 codelab	§16; A documents dry-run in README
Top-3 swag / track reassignment	Wrap-up 709712	§0.1 recognition + track note

1.1.11 0.11 V7.4 revision — operational playbook merge

This revision merges team-operations content from `NightShift_ExecutionPlan-claudeV7.2.md` into V7.3 without changing the **Member A/B/C/D** roster or the approved role split in §2:

Added in V7.4	Source
§2.1 Assumptions & constraints	Claude V7.2 §3
§2.2 Tools & access matrix	Claude V7.2 §4
§3.10 All-member Day 1 onboarding	Claude V7.2 §4
§4.0 Milestones M1–M10	Claude V7.2 §6
§6.1 Git & code review policy	Claude V7.2 §7
§6.2 Escalation path	Claude V7.2 §8
§6.3 Shared resources	Claude V7.2 §8
§10 Testing & validation playbook	Claude V7.2 §10
Expanded §8 submission micro-checks	Claude V7.2 §5–6, §11
Additional §9 risk mitigations	Claude V7.2 §9
Hard-case canonical ID <code>email-001</code> + pytest alias	§3.7

Not adopted from Claude V7.2: named personal roster (kept A/B/C/D); incorrect weekday labels (V7.4 uses correct 2026 calendar: Jul 1 = Wednesday).

1.2 1. Executive Summary

NightShift is an overnight property-management assistant that ingests multi-source communications, classifies urgency (RED/YELLOW/GREEN), drafts responses, and presents a ranked **Morning Brief** — with a non-negotiable guarantee: **drafts never send without explicit human approval, enforced at the database layer.**

This execution plan splits the build into four parallel workstreams that converge on a single demo-ready repo and four mandatory Kaggle submission assets.

North-star demo moment (build this early, reuse everywhere): the ambiguous water-stain fixture from PRD \$1.5 must classify as RED, surface reasoning in the Morning Brief, and show a draft sitting in `staged` until the manager clicks Approve.

1.2.1 The three things that will actually win or lose this

Everything else in this document supports these three priorities:

1. **Build the HITL state machine on Day 1, not Day 3.** It is the headline differentiator. If the `CHECK` constraint is not in the code, *“the database won’t let it send”* is just a claim.
2. **The water-stain case must classify RED.** Tune prompts until it does. This single demo moment carries more weight in the video than ten minutes of easy-case classifications.
3. **Submit on Day 6 before 6:00 PM PT, not at 11:59 PM.** One submission per team, no grace period, draft writeups do not count.

1.2.2 Vertical-slice philosophy

Each member owns a **vertical slice** aligned to TDD \$2.8 Build Priority Order — not a horizontal layer that blocks others. Nobody should wait for a complete “foundation” before starting agent work. Shared contracts in \$3 are frozen on Day 1 so parallel work stays compatible.

```
flowchart TB
    subgraph memberA [Member A - Platform and Ingestion]
        Supervisor[SupervisorNode]
        Ingestion[IngestionAgent]
        MCP[MCP Mock Servers]
    end
    end
    subgraph memberB [Member B - Triage and Evals]
        Triage[TriageAgent]
        Memory[Memory Store]
        Evals[eval_urgency.py]
    end
    end
    subgraph memberC [Member C - Safety and Manager UX]
        StagingDB[(Staging DB + HITL FSM)]
        Response[ResponseAgent]
        Brief[Morning Brief UI]
    end
    end
    subgraph memberD [Member D - Quality and Submission]
        OTel[OpenTelemetry]
        Security[Guardrails]
        Docs[README Writeup Video]
    end
    end
    MCP --> Ingestion
    Supervisor --> Ingestion --> Triage --> Response
    Memory --> Triage
    Memory --> Response
    Response --> StagingDB
    StagingDB --> Brief
    Triage --> Evals
    Supervisor --> OTel
    Triage --> OTel
    Response --> OTel
```

1.3 2. Team Roles & Ownership

Member	Primary ownership	Secondary support
A	ADK supervisor graph, IngestionAgent, MCP mock servers, RawItem contracts, repo scaffold	Integration testing of full pipeline
B	TriageAgent, ClassificationSkill, memory lookup, fixtures, eval_urgency.py, Gherkin urgency specs	Hard-case fixture + confusion matrix
C	Staging DB, HITL state machine, ResponseAgent, Morning Brief, Approve/Edit/Reject/Snooze	End-to-end demo script
D	OpenTelemetry, security guardrails, README, Kaggle writeup, YouTube video, submission checklist	Optional Dockerfile / Cloud Run

Integration owner: rotate daily — A (Jul 1) → B (Jul 2) → C (Jul 3) → D (Jul 4) → all hands (Jul 5–6).

Final integration & Kaggle portal submit: Member A owns the last technical regression + repo hygiene; Member D owns attaching all four mandatory assets and clicking Submit.

1.3.0.1 Alternate role mapping (Claude PDF reference)

The companion PDF assigns roles by agent vertical rather than platform layer. Use **this document’s table above** as the source of truth; the PDF mapping is included so nothing is lost when cross-reading:

Person	PDF role	PDF ownership	This plan’s owner
A	Tech Architect	Supervisor graph, ADK wiring, final integration, Kaggle submission	A (supervisor + MCP + ingestion) + final repo tag
B	Agent Developer	<code>IngestionAgent</code> + all MCP tools + mock fixtures	A (MCP/ingestion) + B (fixture content for evals)
C	Agent Developer	<code>TriageAgent</code> + memory + <code>ResponseAgent</code>	B (triage/memory) + C (response/HITL/brief)
D	Content + QA	Gherkin specs, eval harness, Writeup, video, README	D (all) + B (<code>eval_urgency.py</code>)

1.3.0.2 Alternate role mapping (Copilot V1.0 reference)

Person	Copilot role	Copilot ownership	This plan’s owner
A	Architect	Repo structure, supervisor stub, Pydantic, OTel, Docker, final regression	A
B	MCP / Ingestion	Fixtures, MCP auth, server endpoints, <code>IngestionAgent</code>	A (MCP/ingestion) + B (eval fixtures)
C	Triage / Response / DB	SQLite, Draft table, <code>TriageAgent</code> , <code>ResponseAgent</code> , Morning Brief	B (triage) + C (DB/response/brief)
D	Content / QA	Gherkin, eval harness, README, Writeup, video	D + B (<code>eval_urgency.py</code>)

1.3.1 2.1 Assumptions & constraints

Assumptions (verify Day 1 AM):

Assumption	Owner to confirm
All four members have active Kaggle accounts with phone verification completed	A
Each member has a Google AI Studio account with Gemini API access in their region	All
Each member uses Python 3.11+ and Cursor (or equivalent agentic IDE)	All
<code>google-adk</code> and <code>agents-cli</code> install cleanly via pip in each environment	A
Team has access to a shared public GitHub repo before Jul 5	A
No real tenant data is used — all fixtures are synthetic mock data	All
Gemini rate limits do not block dev — if they do, add a rate limiter in <code>SupervisorNode</code>	A

Constraints (binding):

Constraint	Implication
Hard deadline: July 6, 2026, 11:59 PM PT	One submission per team; no post-deadline iteration
Submit target: July 6, 6:00 PM PT	6-hour buffer for Kaggle platform issues
License: CC-BY-4.0 in repo from Day 1	Required if you win; full legal text in <code>LICENSE</code>
Max team size: 5	This plan uses 4 — no additions after Kaggle Team Merger Deadline
Writeup: ≤2,500 words (target 1,800–2,100)	Track word count daily from Day 2 (Member D)
Video: ≤5 minutes, Public on YouTube	Not Unlisted — judges must access without special permissions
API keys: zero tolerance	No key in any committed file or git history — ever
No competition dataset provided	Mock MCP endpoints are the correct approach, not a placeholder
Winning submission	Source must be reproducible and deliverable under CC-BY-4.0

1.3.2 2.2 Tools & access matrix

Tool	Purpose	Who needs access
Python 3.11+	Primary language	All
Google ADK 2.0 (<code>google-adk</code>)	Agent framework	A, B, C
<code>agents-cli</code>	Run/test agents locally	A, B, C
Gemini API (Google AI Studio)	LLM backend	A, B, C
FastAPI + uvicorn	MCP mock server	A
SQLAlchemy + SQLite	Staging DB + HITL FSM	C
Pydantic v2	Shared contracts (<code>models/core.py</code>)	All
OpenTelemetry SDK	Tracing	D
pytest	Eval + state machine tests	B, D
Docker	Deployment evidence (optional)	A
Cursor IDE	Primary agentic IDE	All
GitHub	Version control; public submission repo	All
Google Docs	Shared Writeup drafting	D (owner), All (review)
YouTube	Video hosting	D
Kaggle.com	Writeup + four mandatory assets	A (submit), D (prepare)
Discord or Slack	Team channel <code>#nightshift</code>	All

Required credentials (each person — local only, never committed):

- **Gemini API key** from Google AI Studio → `.env` only
- **Kaggle account** with phone verification
- **GitHub account** with write access to the shared repo

Final sign-off before Submit (two people):

- **Member A** — code regression, reproducibility, git hygiene, Kaggle team membership
- **Member D** — all four submission assets attached, Writeup under word limit, video Public and ≤5 min

1.4 3. Shared Contracts (do not change without team sync)

All members implement against these interfaces from TDD §2.2.1. **No member should pass loose dicts between agents.**

Shared module: Member A creates `models/core.py` (package layout per Copilot starter) on Day 1. Everyone imports `RawItem`, `ClassifiedItem`, and `Draft` from `models.core` — no one invents local data shapes. A flat `models.py` at repo root is acceptable if the team prefers, but pick one layout Day 1 and freeze it.

1.4.1 3.1 Pydantic models (models/core.py)

raw_text deliberately lives only on RawItem, never copied onto ClassifiedItem or Draft — field-level enforcement of context pruning (TDD §2.3).

```
class RawItem(BaseModel):
    id: str
    source: Literal["email", "hoa_portal", "invoice"]
    tenant_id: str | None
    raw_text: str
    received_at: datetime

class ClassifiedItem(BaseModel):
    id: str
    raw_item_id: str
    urgency_tier: Literal["RED", "YELLOW", "GREEN"]
    property_id: str
    summary: str
    classified_at: datetime

class Draft(BaseModel):
    id: str
    classified_item_id: str
    draft_text: str
    status: Literal["staged", "approved", "rejected", "snoozed", "ready_to_send"]
    approved_by: str | None
    approved_at: datetime | None
```

1.4.2 3.2 MCP tool contracts (Member A owns implementation)

Tool	Input	Output	Mock backing
read_inbox	since: datetime \ None	list[RawItem]	mcp/fixtures/inbox.json
read_hoa_portal	since: datetime \ None	list[RawItem]	mcp/fixtures/hoa_portal.json via local REST stub
read_invoices_folder	none	list[RawItem]	mcp/fixtures/invoices/*.pdf

- Bearer token on every MCP call: Authorization: Bearer <token> (dev placeholder: dev-token-placeholder)
- MCP server implementation: minimal FastAPI or Flask app with bearer-token validation
- All MCP tools are read-only in v1
- Mock fixtures should be realistically messy (typos, garbled HOA notices, run-on sentences) per TDD §2.4 — not textbook-clean prose
- MCP tool descriptions must be meticulous (course Day 2): each tool’s docstring names parameters, types, and constraints — the LLM calls tools based on descriptions
- Error handling: failed MCP calls return structured error messages so agents can self-correct (one bounded retry per TDD §2.2)
- Read-only tools defend against confused-deputy risk (course Day 2): no write/send capability at the tool layer

1.4.3 3.3 HITL state machine (Member C owns enforcement)

```

draft_created → staged → approved → ready_to_send
                ↓
                rejected
                ↓
                snoozed → staged

```

- No `staged → ready_to_send` transition
- `approved` requires non-null `approved_by` and `approved_at`
- Enforced via SQLAlchemy `CHECK` constraint + FSM validator (TDD §2.5)

Explicit FSM transition table (implement in `db/fsm.py` or alongside `db/models.py`):

```

VALID_TRANSITIONS = {
    "staged": {"approved", "rejected", "snoozed"},
    "approved": {"ready_to_send"},
    "snoozed": {"staged"},
    "rejected": set(),
    "ready_to_send": set(),
}
# No staged → ready_to_send; approved requires approved_by + approved_at NOT NULL

```

1.4.4 3.4 Directory scaffold (Member A creates on Day 1)

```

nightshift/
├── README.md
├── LICENSE # CC-BY-4.0 (Day 1)
├── AGENTS.md # cross-tool project rules (course Day 5)
├── pyproject.toml # or requirements.txt – see §3.5
├── .gitignore
├── .env.example
├── main.py # ADK supervisor graph entrypoint
├── agents/
│   ├── ingestion/
│   │   ├── ingestion_agent.py
│   │   └── ingestion_skill/
│   │       ├── SKILL.md
│   │       └── mcp_client.py # MCP tool wrappers (httpx)
│   ├── triage/
│   │   ├── triage_agent.py
│   │   └── classification_skill/
│   │       ├── SKILL.md
│   │       └── prompts.py
│   └── response/
│       ├── response_agent.py
│       └── response_drafting_skill/
│           ├── SKILL.md
│           └── prompts.py
├── mcp/
│   ├── server.py # FastAPI MCP mock server
│   ├── auth.py # bearer token validation
│   └── fixtures/
│       ├── inbox.json # includes water-stain hard case (tenant-123)
│       ├── hoa_portal.json
│       └── invoices/*.pdf
├── memory/
│   ├── store.py # resolve_property_id, get_property_personality
│   └── data/
│       ├── tenant_property_map.json
│       └── property_personality.json
├── models/
│   └── core.py # RawItem, ClassifiedItem, Draft (Pydantic)
├── db/
│   ├── engine.py # SQLite engine + SessionLocal
│   ├── models.py # DraftRow + CHECK constraint
│   └── fsm.py # validate_transition()
├── features/
│   ├── urgency_classification.feature
│   └── hitl_state_machine.feature
├── policy/
│   └── check_no_send.py # SDD: static check – no code path sets status=sent
├── session/
│   └── run_state.py # ADK session + per-run progress (SQLite-backed)
├── tests/
│   ├── eval_urgency.py
│   ├── test_state_machine.py
│   └── fixtures/
│       └── inbox_test.json # mirrors MCP shape + expected_urgency_tier
├── observability/
│   ├── tracing.py # OpenTelemetry spans
│   └── logging_config.py

```

```
└─ docker/
   └─ Dockerfile
   └─ cloudrun.yaml # optional (TDD §2.6)
```

1.4.5 3.5 Environment & dependencies (Member A — Day 1)

`.env.example` (never commit real keys; copy to `.env` locally):

```
GEMINI_API_KEY=your-gemini-key-here
MCP_SERVICE_TOKEN=dev-token-placeholder
MCP_BASE_URL=http://localhost:8000
DB_URL=sqlite:///nightshift.db
OTEL_EXPORTER_ENDPOINT=http://localhost:4317
```

Core dependencies (`pyproject.toml` or `requirements.txt`):

Package	Purpose
<code>google-adk ≥ 2.0</code>	Required — multi-agent supervisor graph (capstone Key Concept #1)
<code>pydantic ≥ 2.0</code>	Shared data contracts
<code>sqlalchemy ≥ 2.0</code>	Staging DB + HITL constraints
<code>fastapi</code> , <code>uvicorn</code>	MCP mock server
<code>httpx</code>	<code>IngestionAgent</code> MCP client
<code>google-genai</code>	Gemini Flash (triage) + Pro (drafting)
<code>opentelemetry-sdk</code> , <code>opentelemetry-exporter-otlp</code>	Tracing (Member D)
<code>pytest</code>	Eval harness + state machine tests

Configure pytest to discover `eval_urgency.py`: `python_files = "test*.py eval_urgency.py"`.

1.4.6 3.6 Memory store contracts (Member B — Day 1)

`memory/data/tenant_property_map.json` — deterministic lookup, no LLM for matching:

```
[
  {
    "tenant_id": "tenant-123",
    "tenant_email": "tenant123@example.com",
    "property_id": "property-A",
    "lease_start": "2025-01-01",
    "lease_end": "2026-12-31"
  }
]
```

`memory/data/property_personality.json` — warm drafts without forwarding `raw_text`:

```
[
  {
    "property_id": "property-A",
    "note": "Older building, high-maintenance HVAC, tenant has been responsive in the past."
  }
]
```

```
memory/store.py API: resolve_property_id(tenant_email_or_id) → str | None ;
get_property_personality(property_id) → str | None .
```

1.4.7 3.7 MCP fixture & eval shape (Member A + B — Day 1)

Hard-case fixture in `mcp/fixtures/inbox.json` (named, reusable in demo + video):

```
{
  "id": "email-001",
  "source": "email",
  "tenant_id": "tenant-123",
  "raw_text": "The ceiling above my bathroom has a small water stain, has had it for a week, nothing dripping yet.",
  "received_at": "2026-06-29T23:15:00Z"
}
```

Eval fixtures in `tests/fixtures/inbox_test.json` use the same shape plus `expected_urgency_tier` per row (hard case = `"RED"`). Reuse MCP JSON shape — do not invent a second format.

Hard-case naming (canonical):

Name	Where used
<code>email-001</code>	Fixture <code>id</code> in <code>mcp/fixtures/inbox.json</code> and eval JSON
<code>tenant-123</code>	<code>tenant_id</code> on the hard case
<code>hard_case_water_stain</code>	pytest alias only — e.g. <code>@pytest.mark.parametrize</code> or test function name; maps to <code>email-001</code>

```
# Hard-case regression (canonical fixture id)
pytest tests/eval_urgency.py -k "email-001 or hard_case_water_stain" -v
```

MCP HTTP endpoints (FastAPI): `GET /read_inbox`, `GET /read_hoa_portal`, `GET /read_invoices_folder` — all require `Authorization: Bearer dev-token-placeholder`.

1.4.8 3.8 Agent Skills template (course Day 3 — progressive disclosure)

Each skill folder uses YAML frontmatter so hosts/agents can load skills on demand (not all at startup):

```
---
name: classification-skill
description: Classify overnight property items as RED, YELLOW, or GREEN urgency
---

# Classification Skill
## Overview
## Workflow
## Steps
```

- Keep each SKILL.md under 500 lines (course best practice)
- Optional: scripts/ and examples/ subfolders per skill
- Progressive disclosure: system prompts stay lightweight; skill details load when the agent needs that capability

1.4.9 3.9 Repo governance files (course Day 5)

File	Purpose	Owner	Day
AGENTS.md	Cross-tool project rules (constraint harness): coding standards, no sent bypass, spec-first workflow	D	1
features/*.feature	Gherkin BDD — source of truth; code is disposable	D	1
policy/check_no_send.py	Hybrid policy: structural check for forbidden sent transitions	D	4
scripts/pre_submit.sh (optional)	Runs policy check + pytest + key scan before tag	D	5

1.4.10 3.10 Environment onboarding (all members — Day 1 AM)

Each member runs this on their local machine before writing code:

```

# 1. Clone the repo
git clone https://github.com/<your-org>/nightshift.git
cd nightshift

# 2. Create virtual environment
python3 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt
# or: pip install google-adk pydantic fastapi uvicorn sqlalchemy httpx python-dotenv pytest opentelemetry-
      sdk

# 4. Set up environment variables (never commit the real .env)
cp .env.example .env
# Then edit .env and add your own Gemini API key locally

# 5. Verify ADK / agents-cli
python main.py --dry-run
agents-cli --version # if installed; document equivalent if using python main.py only

# 6. Init DB and run smoke tests (after Member C lands Day 1 scaffold)
python db/init_db.py
pytest tests/test_state_machine.py -q

```

Optional Day 1 (Member A): add `.pre-commit-config.yaml` with a hook that rejects common API key patterns in staged files — complements `scripts/pre_submit.sh` on Day 5.

1.5 4. Day-by-Day Schedule

Aligned to TDD §2.8 build priority order. Each day ends with a **named integration checkpoint** — do not start the next day's scope until the checkpoint passes.

1.5.1 4.0 Milestones at a glance (M1–M10)

Milestone	Date	Owner	Definition of done
M1	Jul 1 EOD	A	Repo public; <code>python main.py --dry-run</code> passes for all members
M2	Jul 1 EOD	C	Staging DB live; rejects <code>status="sent"</code> and invalid FSM transitions
M3	Jul 1 EOD	B	20+ fixture files in <code>mcp/fixtures/</code> ; hard case <code>email-001</code> present
M4	Jul 2 EOD	A + B	First real classifications; easy RED/GREEN items correct
M5	Jul 3 EOD	A	First full E2E batch; all drafts <code>staged</code>
M6	Jul 3 EOD	B	Hard case <code>email-001</code> → RED with visible reasoning trace
M7	Jul 4 EOD	D	Video recorded ≤5 min; all five PRD beats covered
M8	Jul 5 AM	B + D	Triage accuracy ≥90%; confusion matrix reviewed
M9	Jul 5 EOD	A	Fresh-clone reproducibility test passes; no key leaks in git history
M10	Jul 5 EOD	D	All four Kaggle assets attached in portal UI
SUBMIT	Jul 6 by 6 PM PT	D (A co-signs technical)	Writeup status = Submitted in Kaggle

July 4 buffer: US holiday — treat **Jul 3 EOD** as the deadline for core E2E (M5–M6). Jul 4 is polish, security, and video — not net-new architecture.

1.5.2 Schedule at a glance (Gantt summary)

Day	Date	A	B	C	D
1	Jul 1	Repo, <code>db/</code> , supervisor stub, MCP skeleton	Memory JSON + <code>store.py</code> , fixtures	<code>DraftRow</code> + CHECK + FSM + <code>test_state_machine.py</code>	Gherkin (2 features), eval skeleton, Writeup outline
2	Jul 2	Real ingestion, failure isolation, <code>ThreadPoolExecutor</code>	<code>TriageAgent</code> Flash + memory lookup	<code>ResponseAgent</code> → <code>staged</code> , <code>triage_failed</code>	OTel <code>tracing.py</code> , README headline
3	Jul 3	E2E batch, Morning Brief CLI	MCP verify, README <code>mcp/</code> section	DB guarantee test, personality notes, manager CLI	Eval confusion matrix, Writeup draft
4	Jul 4	Invoice audit tool, Docker + health-check	Eval ≥90%, Gherkin pass	Blue/Green validation, brief polish	Video record, Writeup peer review
5	Jul 5	Regression, git scan, clean-clone test	README verify (another member)	DB integrity screenshot	YouTube Public, Kaggle assets
6	Jul 6	Team merge, final tag	—	—	Submit by 6 PM PT
7	Jul 7	Buffer only (optional internal polish)	—	—	—

1.5.3 Day 1 — Wednesday, July 1: Foundation & Plumbing

Goal: Runnable supervisor graph with stub agents; repo scaffold; team contracts frozen.

Time block	Member A	Member B	Member C	Member D
AM	Create public GitHub repo + <code>LICENSE</code> + <code>AGENTS.md</code> scaffold	Draft <code>memory/tenant_property_map.json</code> schema	Design SQLAlchemy models for <code>Draft</code> + FSM validator sketch	Read PRD + TDD + Learn Guide Day 1 ; Writeup outline in Google Doc
PM	<code>models/core.py</code> + ADK 2.0 supervisor graph stub in <code>main.py</code> ; <code>db/engine.py</code>	<code>memory/data/</code> JSON + <code>memory/store.py</code> ; hard case <code>tenant-123</code>	<code>db/models.py</code> <code>DraftRow</code> + CHECK + <code>db/fsm.py</code> (before agents write)	Gherkin: <code>urgency_classification.feature</code> + <code>hitl_state_machine.feature</code>
EOD	<code>mcp/server.py</code> + <code>mcp/auth.py</code> ; <code>read_inbox</code> live; python <code>main.py --dry-run</code>	Messy fixtures (~20); <code>SKILL.md</code> stubs for triage skill	<code>tests/test_state_machine.py</code>	<code>eval_urgency.py</code> skeleton; <code>policy/check_no_send.py</code> stub

Day 1 integration checkpoint (Owner: A): - [] `python main.py --dry-run` executes supervisor graph end-to-end with stub agents - [] `read_inbox` returns at least one `RawItem` including the water-stain hard case - [] Staging DB rejects `staged → ready_to_send` in a unit test - [] `.env.example` documents `GEMINI_API_KEY`, `MCP_SERVICE_TOKEN`, `MCP_BASE_URL`, `DB_URL`, `OTEL_EXPORTER_ENDPOINT` — no real keys in repo - [] `LICENSE` (CC-BY-4.0) present in repo root - [] `models/core.py` exists; all agents import shared schemas - [] **ADK 2.0** graph visible in `main.py` with three named sub-agents (not one monolithic agent) - [] `google-adk` in `requirements.txt` / `pyproject.toml` - [] FSM rejects transition to `approved` without non-null `approved_by` and `approved_at` (DB CHECK and/or validator) - [] All members completed \$3.10 onboarding script successfully

1.5.4 Day 2 — Thursday, July 2: Ingestion + Triage Live

Goal: Real ingestion from all three MCP sources; live classification with memory lookup.

Time block	Member A	Member B	Member C	Member D
AM	Complete MCP tools; wire ADK IngestionAgent into supervisor graph	Implement ADK TriageAgent + <code>ClassificationSkill</code> + <code>SKILL.md</code>	Wire <code>ClassifiedItem</code> to DB; begin ADK ResponseAgent	OTel spans; README safety headline
PM	<code>IngestionAgent</code> calls all three MCP tools; dedupe by <code>id</code>	Memory lookup: <code>deterministic tenant_id → property_id</code>	Per-item failure isolation (coordinate with A)	Red-team content scan stub
EOD	ADK session per overnight run + <code>session/run_state.py</code> ; concurrency 3–5 in flight	20+ eval fixtures; hard case → RED; <code>agents-cli</code> test or <code>pytest</code> documented	<code>triage_failed</code> in brief placeholder	OTel: MCP tool latency spans

Day 2 integration checkpoint (Owner: B): - [] Overnight batch of 10+ items ingested from all three sources - [] Hard-case water stain classifies RED with logged reasoning - [] Memory lookup resolves tenant → property for ≥ 98% of tenant-tied fixtures - [] One injected failure does not crash the batch; item flagged `trriage_failed` - [] OTel trace shows full path for one RED item - [] ADK session initialized once per batch; run state recoverable after restart

1.5.5 Day 3 — Friday, July 3: Integration + First End-to-End Run

Goal: First full overnight batch; ResponseAgent drafts to `staged`; Morning Brief ranks RED first; manager can Approve.

Time block	Member A	Member B	Member C	Member D
AM	Run first full E2E batch; fix integration bugs; minimal Morning Brief (CLI OK)	Verify all three MCP tools with real <code>IngestionAgent</code> ; fix contract mismatches	Prove the guarantee: attempt <code>Draft(status="sent")</code> — DB must reject; screenshot result for Writeup	Run eval harness on real <code>TriageAgent</code> output; share confusion matrix
PM	Morning Brief assembly (no extra LLM call)	<code>mcp/</code> README section; memory consolidation script (no mid-run writes)	Manager CLI: Approve / Edit / Reject / Snooze; <code>ResponseDraftingSkill/SKILL.md</code>	Writeup draft + hard case; circulate for fact-check
EOD	End-to-end run: ingest → classify → draft → brief	Hand off MCP README section	Manager UI/CLI: Approve / Edit & Approve / Reject / Snooze	Plan video recording setup (live run vs step-through)

Day 3 integration checkpoint (Owner: C): - [] Full pipeline produces Morning Brief with RED items first - [] Every draft has `status=staged`; no outbound send code path exists - [] Manager Approve transitions `staged → approved` with identity + timestamp - [] Direct `status="sent"` write rejected by DB — test result documented/screenshoted - [] `eval_urgency.py` runs green on hard case; confusion matrix generated - [] Time-to-brief < 10 min for 50-item batch (or documented gap + fix plan)

1.5.6 Day 4 — Saturday, July 4: Quality, Security & Hardening

Goal: Eval suite stable; security guardrails documented; optional sandboxed invoice audit.

Time block	Member A	Member B	Member C	Member D
AM	Invoice-audit code-execution tool (restricted function OK per TDD §2.8 fallback)	Expand fixtures: messy typos, garbled HOA notices	Edit & Approve flow; snooze → re-queue	README sections 1–6 (PRD §3.4)
PM	Lease-date cross-reference tool	Target ≥90% eval; <code>agents-cli test</code> or <code>pytest</code> green	Brief shows <code>trriage_failed</code> distinctly	<code>policy/check_no_send.py</code> enforced in CI/local script
EOD	Docker + health-check endpoint ; confirm <code>docker build</code> && <code>docker run</code> locally	Gherkin scenarios pass via test runner	Record screen captures for video hard-case segment; prepare before/after split-screen assets	Kaggle writeup draft (~1,800–2,100 words); 2 members peer-review against actual code

Day 4 integration checkpoint (Owner: D): - [] `pytest tests/` and `eval_urgency.py` pass in CI or local script - [] Confusion matrix reviewed; YELLOW/GREEN confusion documented if present - [] README clone-to-run steps verified on a clean machine by one non-author member - [] Writeup draft covers architecture diagram, hard case, Key Concepts (5/6) - [] `git log` and repo scanned for API keys (PRD §0, §3.5)

1.5.7 Day 5 — Sunday, July 5: Demo Polish & Submission Assets

Goal: Memorable demo; all four submission assets draft-complete.

Time block	Member A	Member B	Member C	Member D
AM	Full regression: <code>pytest tests/eval_urgency.py</code> ≥90% accuracy; <code>git log -p</code> key scan	Re-run evals after any prompt changes	DB integrity check: zero <code>sent</code> rows after full batch — screenshot for Writeup	Finalize README + architecture diagram image
PM	Clean-clone reproducibility test on fresh machine/dir (required if you win)	README setup verified by another member following steps verbatim	ResponseAgent draft quality: 5 drafts warm/specific (personality notes)	Record YouTube video (≤5 min, PRD §3.2)
EOD	Tag <code>v1.0.0-rc</code> ; paste Writeup to Kaggle (diagram + GitHub link)	Add Scaling Notes to README (TDD §2.6)	Sign off demo script timing (~2 min hard-case segment)	Upload video Public ; cover image; attach all four assets

Day 5 integration checkpoint (All hands): - [] Full demo run recorded; reasoning trace displays cleanly (caption OK per PRD §3.2) - [] Public GitHub repo link works; README setup verified - [] YouTube video uploaded (unlisted → public before submit) - [] Writeup ≤ 2,500 words; track stated: **Agents for Business** - [] Media Gallery cover image attached

1.5.8 Day 6 — Monday, July 6: Submit

Goal: Click **Submit** before 6:00 PM PT (hard deadline 11:59 PM PT).

Time (PT)	Activity	Owner
9:00 AM	Final <code>pytest</code> + eval run on release tag	B
10:00 AM	Fresh clone setup test on clean env	D
11:00 AM	Merge all team members in Kaggle before deadline	A
12:00 PM	CODE FREEZE — only submission-field edits after this	All
2:00 PM	Kaggle Writeup status = Submitted (not draft)	D
4:00 PM	Verify all four assets attached: Writeup, cover image, YouTube, GitHub link	D
6:00 PM	Click Submit — primary target, 6-hour buffer remaining	D
6:00 PM	Final repo tag <code>v1.0.0-submission</code> ; no further commits	A

1.5.9 Day 7 — Tuesday, July 7: Buffer only (optional)

Not a build day — submission deadline is July 6. Use Day 7 only for post-submit bugfixes if Kaggle allows edits (official rules: **single submission** — treat Day 7 as internal polish only, not a second submit window). If all Day 6 checkpoints pass, skip Day 7 entirely.

Activity	Owner
Internal doc tweaks, post-mortem notes	D
Non-submission bugfixes on a branch (do not retag submission)	All

1.6 5. Member Workstreams (Detailed)

1.6.1 5.1 Team Member A — Platform, Supervisor & MCP Ingestion

Mission: Own the graph topology judges inspect in code — three distinct ADK agents coordinated by a non-LLM `SupervisorNode`.

1.6.1.1 Deliverables

#	Deliverable	File(s)	Depends on	Done when
A0	Public repo + CC-BY-4.0 license	<code>LICENSE</code> , GitHub settings	—	Repo public; license file present Day 1
A1	Repo scaffold + dependencies	<code>pyproject.toml</code> or <code>requirements.txt</code> , <code>.env.example</code> , <code>models/core.py</code> , <code>main.py</code>	—	Clean clone + install works
A1b	DB layer scaffold	<code>db/engine.py</code> , <code>db/models.py</code> , <code>db/fsm.py</code>	A1	SessionLocal + DraftRow CHECK constraint
A2	ADK 2.0 supervisor graph + stub agents	<code>main.py</code> , <code>agents/*/</code>	A1	Three ADK sub-agents visible; graph runnable
A2b	ADK session per overnight run	<code>session/run_state.py</code> , ADK config	A2	One session/batch; SQLite run state reconstructable
A3	MCP mock server + three read tools	<code>mcp/</code> , <code>mcp/fixtures/</code>	A1	All tools return valid <code>RawItem</code> lists
A4	<code>IngestionAgent</code> + <code>IngestionSkill</code> + <code>SKILL.md</code>	<code>agents/ingestion/</code>	A2, A3	Produces deduped <code>RawItem</code> stream
A4b	<code>IngestionSkill/SKILL.md</code>	skill folder	A4	Documents MCP ingestion capability
A5	Per-item concurrency (3–5 in flight)	<code>main.py</code> SupervisorNode	A2	50-item batch completes in < 10 min
A6	Failure isolation + <code>trriage_failed</code> flag	SupervisorNode try/except	A2, C3	One bad item does not halt batch
A7	Sandboxed invoice audit tool (fallback OK)	<code>agents/triage/tools/</code> or shared	B2	Line-item sum vs total returns structured result
A8	Dockerfile + health-check endpoint	<code>Dockerfile</code>	All core	<code>docker build &&</code> <code>docker run</code> succeeds; health endpoint responds
A9	Final integration + Kaggle submit checklist	release tag, portal	All	Regression green; four assets attached; Submit clicked

1.6.1.2 Key implementation notes

- SupervisorNode is **plain Python routing** — no LLM call (TDD §2.2)
- Process items individually for context cleanliness; concurrency is a scheduling concern only
- Bearer token on MCP calls even with mock token in dev
- Include hard-case data flow comment at `main.py` entrypoint (TDD §2.2.1)

1.6.1.3 Definition of done (Member A)

- Judge can open `main.py` and see **three ADK sub-agents** (not one agent with internal functions)
- Runnable via `agents-cli run` or documented equivalent in README
- MCP swap-in story is true: changing `read_inbox` backend does not touch triage/response
- No API keys in code or git history
- Day 1 and Day 2 integration checkpoints pass

1.6.2 5.2 Team Member B — Triage, Memory & Evaluation

Mission: Own classification accuracy, deterministic property matching, and the regression harness that protects the hard case.

1.6.2.1 Deliverables

#	Deliverable	File(s)	Depends on	Done when
B1	Memory store + tenant→property map	<code>memory/store.py</code> , <code>memory/data/</code>	A1	Lookup returns <code>property_id</code> or explicit miss
B1b	Nightly memory consolidation	<code>memory/consolidate.py</code>	B1	Agents read-only mid-run; writes only via consolidation
B2	ADK <code>TriageAgent</code> + <code>ClassificationSkill</code> + <code>SKILL.md</code>	<code>agents/triage/</code>	A4, B1	Emits <code>ClassifiedItem</code> ; prunes <code>raw_text</code>
B3	Hard-case fixture (named, RED ground truth)	<code>mcp/fixtures/inbox.json</code> (<code>email-001</code>), <code>tests/fixtures/</code>	A3	Canonical id <code>email-001</code> ; pytest alias <code>hard_case_water_stain</code>
B4	20+ eval fixtures (messy, realistic)	<code>tests/fixtures/</code>	B2	6–8 RED, 6–8 GREEN, 4–6 YELLOW, 1 hard
B5	<code>eval_urgency.py</code> + confusion matrix	<code>tests/eval_urgency.py</code>	B4	Reports accuracy, false-RED, false-GREEN
B6	Gherkin specs	<code>features/urgency_classification.feature</code> , <code>features/hitl_state_machine.feature</code>	B2, C2	TDD §2.5 scenarios + HITL transitions executable
B8	<code>test_state_machine.py</code>	<code>tests/test_state_machine.py</code>	C2	Invalid transitions raise; <code>staged→ready_to_send</code> blocked
B7	Property-context for ResponseAgent	<code>memory/</code> personality notes	B1, C2	Drafts can be warm without raw text forward

1.6.2.2 Eval fixture composition (TDD §2.4)

Category	Count	Examples
Unambiguous RED	6–8	active leak, no heat, city violation notice
Unambiguous GREEN	6–8	lightbulb out, routine HOA newsletter
Unambiguous YELLOW	4–6	invoice mismatch, inspection in 5 days
Hard case (RED)	1	water stain, no drip, one week

1.6.2.3 Success metrics owned by B (PRD §1.3)

Metric	Target	How measured
Triage accuracy	≥ 90%	<code>eval_urgency.py</code> vs labeled fixtures
False-RED rate	< 15%	Confusion matrix
False-GREEN rate	< 2%	Confusion matrix — critical
Memory lookup success	≥ 98%	Fixtures with <code>tenant_id</code>

1.6.2.4 Definition of done (Member B)

Hard case classifies RED with stated rationale in trace/logs

Confusion matrix attached to README or writeup

No `raw_text` on `ClassifiedItem` or `Draft` objects

Day 2 and Day 3 integration checkpoints pass

1.6.3 5.3 Team Member C — HITL Safety, Drafting & Morning Brief

Mission: Own the headline differentiator — “the database won’t let it send” — plus the manager-facing Morning Brief.

1.6.3.1 Deliverables

#	Deliverable	File(s)	Depends on	Done when
C1	Staging DB schema	db/engine.py , db/models.py	A1b	DraftRow table; optional Alembic in db/migrations/
C2	HITL FSM + CHECK constraints	SQLAlchemy models	C1	Invalid transitions rejected at DB
C3	ADK ResponseAgent + ResponseDraftingSkill + SKILL.md	agents/response/	B2, C1	Writes Draft(status=staged) only
C4	Morning Brief assembly	brief/ or main.py	B2, C3	RED first; includes rationale summary
C5	Manager actions UI or CLI	ui/ or CLI commands	C2	Approve / Edit & Approve / Reject / Snooze
C6	trriage_failed display in brief	brief module	A6	Failed items visible, not dropped
C7	Demo script	docs/demo_script.md	C4, C5	2-min hard-case segment scripted

1.6.3.2 HITL implementation checklist (TDD §2.5)

SQLAlchemy model with CHECK on status enum

FSM validator function rejects illegal transitions before commit

approved requires approved_by + approved_at NOT NULL

No application code path sets ready_to_send without approved

Unit test: agent code cannot bypass DB constraint (attempt direct SQL update)

1.6.3.3 Morning Brief requirements (PRD §1.4 Workflow B)

1. Items grouped by urgency — RED first
2. Per item: summary, draft text (if any), classification rationale
3. Actions: Approve / Edit & Approve / Reject / Snooze
4. triage_failed items flagged: “could not classify — needs manual review”

1.6.3.4 Definition of done (Member C)

Live demo shows draft in staged until manager clicks Approve

PRD Non-Goal enforced: no auto-send even for GREEN

Headline hook demonstrable in < 30 seconds of screen recording

Day 3 and Day 4 integration checkpoints pass

1.6.4 5.4 Team Member D — Observability, Security & Submission

Mission: Make the system auditable, trustworthy, and submittable — README (20 pts), Writeup (10 pts), Video (10 pts).

1.6.4.1 Deliverables

#	Deliverable	File(s)	Depends on	Done when
D1	OpenTelemetry tracing	<code>observability/tracing.py</code> , <code>logging_config.py</code>	A2	Per-item trace with model, tokens, tool latency
D2	Secret/PII redaction in traces	OTel config	D1	No keys or tenant PII in span attributes
D3	Red-team inbound content scan	<code>security/</code>	A4	Injection patterns blocked or flagged
D4	Blue/Green output validation	<code>security/</code>	C3	Cross-tenant leakage caught
D4b	Policy check script (SDD)	<code>policy/check_no_send.py</code>	C2	Fails if any code path sets <code>status=sent</code> or skips <code>approved</code>
D5	README (20 rubric pts)	<code>README.md</code>	All	PRD §3.4; includes <code>agents-cli</code> run instructions
D6	Architecture diagram image	<code>docs/architecture.png</code>	A2	Embedded in README + writeup
D7	Kaggle Writeup	Kaggle portal	All	≤2,500 words; ~1,800–2,100 target
D8	YouTube video ≤5 min	YouTube + Kaggle	C7	Title: <i>"NightShift — An overnight property triage agent that drafts but never sends."</i> Public; PRD §3.2 five beats
D11	Shared Writeup Google Doc	Google Doc	Day 1	Headings Day 1; prose Day 3–5; peer-reviewed by 2 members
D12	<code>AGENTS.md</code> project rules	repo root	Day 1	Constraint harness documented; references Gherkin + policy
D13	<code>scripts/pre_submit.sh</code> (optional)	<code>scripts/</code>	Day 5	policy + pytest + key scan before release tag
D9	Media Gallery cover image	Kaggle	—	Attached with writeup
D10	Submission checklist execution	this doc §8	All	All four mandatory assets submitted

1.6.4.2 README required sections (PRD §3.4)

1. Safety guarantee headline (first lines)
2. Problem (2–3 sentences)
3. Solution overview

- 4. Mock data sources note (early, prominent)
- 5. Architecture diagram
- 6. Setup: clone → `.env` → run MCP → `agents-cli run` or `python main.py`
- 7. Project structure tree
- 8. Optional: deployment / scaling notes (TDD §2.6)
- 9. How to run tests: `agents-cli test` or `pytest tests/`

1.6.4.3 Video script beats (PRD §3.2)

Segment	Duration	Content
Hook + problem	~45 sec	Speak headline; overnight inbox pain; optional before/after split
Why agents	~30 sec	Judgment on ambiguous text; cite Day 4 expense-approval parallel — NightShift is same HITL shape, stricter (no auto-send)
Architecture	~50 sec	Three agents + MCP + memory on screen
Hard-case demo	~2 min	Water stain → RED trace → brief → staged draft → Approve. Scripted on-screen reasoning: <i>"Reasoning: 'small water stain... for a week' matches early structural water-damage pattern (RED). Even without active drip, risk of code-violation escalation within days is high."</i>
The Build	~25–35 sec	ADK, Gemini, MCP, Cursor vs Antigravity mention
Before/after (optional)	~5 sec	Chaotic mock inbox left; ranked Morning Brief right — open hook

1.6.4.4 Definition of done (Member D)

- 100% of classifications traceable via OpenTelemetry (PRD §1.3)
- README setup works from clean clone (verified by non-author)
- All four Kaggle assets submitted before deadline
- No API keys in repo or git history
- Day 4, Day 5, Day 6 checkpoints pass

1.7 6. Dependency Graph & Handoff Points

```
flowchart LR
  A1[A1 Scaffold] --> A2[A2 Graph]
  A2 --> A3[A3 MCP]
  A3 --> A4[A4 Ingestion]
  B1[B1 Memory] --> B2[B2 Triage]
  A4 --> B2
  C1[C1 Staging DB] --> C2[C2 HITL FSM]
  C2 --> C3[C3 Response]
  B2 --> C3
  C3 --> C4[C4 Morning Brief]
  B2 --> B5[B5 Evals]
  A2 --> D1[D1 0Tel]
  C4 --> D8[D8 Video]
  C4 --> D7[D7 Writeup]
```

1.7.1 Critical handoffs (blocking)

From	To	Artifact	Deadline
A	All	Repo scaffold + <code>.env.example</code>	Jul 1, 12:00 PM
A	B	<code>RawItem</code> from <code>read_inbox</code> with hard case	Jul 1, EOD
B	C	<code>ClassifiedItem</code> with urgency + summary	Jul 2, EOD
C	D	Working Approve flow + staged draft	Jul 3, EOD
B	D	Eval metrics + confusion matrix	Jul 4, EOD
All	D	Release candidate tag	Jul 5, EOD

1.7.2 Communication rhythm

- **Daily standup (async):** post in `#nightshift` by 9:00 AM — three lines per person:
 1. What I completed yesterday
 2. What I’m working on today
 3. Any blocker — tag the person who can unblock you
- **Integration hour:** 5:00 PM — integration owner runs checkpoint script
- **Shared doc:** update checkpoint checkboxes in this file’s §4 daily sections

1.7.3 6.1 Git & code review policy

- All merges to `main` via **Pull Request** — no direct pushes to `main`
- Each PR requires **one approving review from Member A** before merge
- PRs touching `models/core.py`, `db/models.py`, or `db/fsm.py` require review from **both A and C** — these files enforce the project’s core guarantees
- No PR merges a new package not listed in `requirements.txt` without a **slopsquatting review** (verify package name on PyPI; pin versions)
- **Optional Day 1:** `.pre-commit-config.yaml` hook scanning for `AIza`, `sk-`, `ghp_`, `AKIA` patterns in staged files

1.7.4 6.2 Escalation path

Situation	Contact	Response time
Blocked on a technical dependency	A	Within 2 hours
Gemini API quota / rate limit	A	Immediately — add supervisor rate limiter
Repo access problem	A	Immediately
Kaggle platform issue	A + D	Immediately — contact Kaggle support
API key accidentally committed	A	Stop all pushes — run <code>git filter-repo</code> before any public push
Hard case classifies GREEN in eval	B + A	Same day — tune prompt; apply fallback rule (§9)

1.7.5 6.3 Shared resources

Resource	Location	Purpose
GitHub repo	<code>github.com/<org>/nightshift</code>	Code, README, LICENSE
Kaggle Writeup draft	Google Docs (shared link)	Collaborative writing — Member D owns
Team channel	Discord or Slack <code>#nightshift</code>	Standups, blockers, checkpoint status
This execution plan	<code>NightShift_Execution_Plan-V7.4.md</code> (+ PDF export)	Source of truth for schedule and checkpoints
Architecture diagram	<code>docs/architecture.png</code>	README + Writeup embed

1.8 7. Rubric & Requirements Traceability

1.8.1 7.1 Kaggle rubric → owner

Rubric item	Points	Primary owner	Evidence location
Core Concept & Value	10	D (writeup) + C (demo)	Writeup §problem; headline in README + video
YouTube Video	10	D	PRD §3.2 script; hard-case segment
Writeup	10	D	PRD §3.1 outline
Technical Implementation	50	A, B, C	TDD §2.1–§2.5; graph in code
Documentation (README)	20	D	PRD §3.4
Total	100		

1.8.2 7.2 Key Concepts checklist (need 3 of 6; target 5 of 6)

Concept	Owner	Evidence
Agent / Multi-agent (ADK)	A	<code>SupervisorNode</code> + 3 agents in <code>agents/</code>
MCP Server	A	<code>mcp/</code> read tools
Security features	D	Red/Blue/Green + HITL (C)
Agent skills / Agents CLI	B, C	<code>skills/</code> per agent; <code>agents-cli</code> run
Deployability	D	Dockerfile (optional); README scaling notes
Antigravity	D	Mentioned in video §3.2 ("built in Cursor")

1.8.3 7.3 TDD build priority → day mapping

TDD §2.8 step	Day	Owner
1. Supervisor + stub agents	Jul 1	A
2. MCP + IngestionAgent	Jul 1–2	A
3. TriageAgent + memory + hard case	Jul 2	B
4. HITL state machine + staging DB	Jul 1–3	C
5. ResponseAgent + Morning Brief	Jul 3	C
6. Observability + eval harness	Jul 2–4	D, B
7. Polish (README, video, Docker)	Jul 4–6	D, A

1.8.4 7.4 PRD success metrics → owner

Metric	Target	Owner	Tool
Triage accuracy	≥ 90%	B	<code>eval_urgency.py</code>
False-RED	< 15%	B	Confusion matrix
False-GREEN	< 2%	B	Confusion matrix
Time-to-Morning-Brief	< 10 min / 50 items	A	OTel + batch timer
Per-item latency	< 5 sec (Flash)	B	OTel span
Memory lookup	≥ 98%	B	Eval fixtures
Draft acceptance	≥ 60%	C	Track in demo (v1: qualitative)
Auditability	100% traced	D	OTel
Token cost	tracked	D	OTel token attributes

1.8.5 7.5 Course Day 5 (SDD + observability) → deliverable checklist

Course requirement	Deliverable	Owner	Done by
Gherkin as source of truth	<code>features/*.feature</code>	D	Jul 1
Policy / hybrid enforcement	<code>policy/check_no_send.py</code>	D	Jul 4
OpenTelemetry audit trail	<code>observability/tracing.py</code>	D	Jul 4
Optional Cloud Run deploy	<code>docker/Dockerfile</code> , <code>cloudrun.yaml</code>	A	Jul 4–5
A2A scope statement	Writeup: internal ADK graph only; A2A reserved for v2	D	Jul 5
Threat model (Effective Trust)	Writeup: 2–3 failure scenarios + HITL backstop	D	Jul 5

1.9 8. Final Submission Checklist

Complete every item before July 6, 11:59 PM PT. **Draft writeups do not count.**

1.9.1 8.1 Code & repo

GitHub repo is **public** — verify in an **incognito/private browser window**, not only while logged in

`LICENSE` file is **CC-BY-4.0** with full legal text (required if you win)

`.env` is **not** tracked: `git ls-files | grep -E '^\.env$'` returns nothing

Public GitHub repo link ready (Public Project Link field)

`README.md` complete per PRD §3.4 — **safety headline is first content after title**

`python main.py` (or documented command) runs overnight batch on mock data

`pytest tests/` and/or `agents-cli test` passes including `eval_urgency.py`

Hard case `email-001` (alias `hard_case_water_stain`) classifies **RED**

`policy/check_no_send.py` passes (no direct `sent` code path)

ADK 2.0 multi-agent graph demonstrable in repo

Three `SKILL.md` files present (ingestion, triage, response)

HITL: DB rejects `staged → ready_to_send` and direct `status="sent"` writes

`approved` transition requires `approved_by` + `approved_at` NOT NULL

`Draft` table `CHECK` constraint visible in SQLAlchemy model

DB integrity screenshot after full batch (see §10.4 SQL query) — zero `sent` rows

No API keys/passwords in current files **or git history** (see §10.3 git scan)

Clean-clone reproducibility test passed (see §10.5)

All dependency licenses compatible with CC-BY 4.0 (if aiming to win)

Tag: `v1.0.0-submission`

1.9.2 8.2 Kaggle portal (four mandatory assets)

All **team members merged** in Kaggle before deadline (Team Merger Deadline)

Track set to **Agents for Business** (one track only)

Qualifying submission → Kaggle **badge + certificate** (verify after submit)

Writeup submitted (not draft) — ≤2,500 words; target 1,800–2,100

Media Gallery — cover image attached (minimum: project name + subtitle; dark background + white text is sufficient)

YouTube video — **Public** (not Unlisted) URL attached, ≤5 minutes, title: *“NightShift — An overnight property triage agent that drafts but never sends.”*

Submit clicked before 6:00 PM PT (not last-minute 11:59 PM)

Public Project Link — repo URL that matches README

1.9.3 8.3 Writeup content checklist (PRD §3.1)

Title: NightShift; subtitle: “drafts overnight, never sends without you”

Track: Agents for Business (stated once)

Factory model framing: Context (memory/MCP) + Constraint (HITL/policy) + Evaluation (Gherkin/pytest) harnesses

Problem with concrete failure modes (fines, structural damage)

Architecture diagram with one-line caption per agent

Workflow A + B summarized

Hard case walkthrough in text

Key Concepts table (5 items)

2–3 rows from TDD §2.7 Architecture Decision Log

A2A not used — one sentence why (internal ADK graph; A2A for external agents in v2; course Day 2)

7-pillar Effective Trust — table or bullets mapping pillars to NightShift (§0.9)

Day 4 HITL parallel — cite expense-approval agent; explain stricter staging guarantee

SDD narrative — Gherkin is source of truth; code is disposable (course Day 5)

Brief **threat model** — MCP read-only + confused deputy + HITL backstop (TDD §2.4)

Engineering journey — 2–3 real tradeoffs (multi-agent vs monolith, SQLite vs Postgres, etc.)

Public project link restated

1.9.4 8.4 Video checklist (PRD §3.2)

Opens with headline spoken verbatim

Hard-case demo ~2 minutes with RED reasoning visible (`email-001`)

Shows staged draft until Approve click

Names ADK, Gemini, MCP

Mentions Antigravity / Cursor equivalence

Total runtime ≤ 5 minutes (~4:30 scripted target; cut Architecture segment first if over)

1.9.5 8.5 Security final pass

`.env` in `.gitignore`; `.env.example` has placeholders only

OTel redaction list applied (keys + tenant PII)

MCP tools read-only confirmed

Slopsquatting / dependency review on `requirements.txt`

`AGENTS.md` present at repo root

`policy/check_no_send.py` passes

1.10 9. Risk Register & Mitigations

Risk	Impact	Owner	Mitigation
Plain Python graph without ADK	Fails Key Concept #1 (50 pts at risk)	A	Use <code>google-adk</code> from Day 1; §15 patterns are illustrative only
Hard case classifies GREEN	Loses differentiator	B	Pin prompt + fixture early; eval blocks regressions
Time-to-brief > 10 min	Misses PRD metric	A	Concurrency 3–5; profile OTel spans Day 3
HITL built after ResponseAgent	Safety gap during build	C	TDD §2.8 order: FSM before drafting (Day 1–3)
Sandbox isolation blocks progress	Delays triage tools	A	Use restricted function fallback (TDD §2.8)
README setup fails for judge	Loses 20 pts	D	Clean-clone test Day 4 by non-author
API key in git history	Disqualification	All	<code>git log -p</code> scan Day 4; use <code>git filter-repo</code> if needed
Video live model output messy	Weak demo	D	Use scripted on-screen caption (PRD §3.2)
Scope creep (real Gmail, Cloud Run)	Misses deadline	All	Stretch goals only after Day 4 checkpoint green
Hard case classifies GREEN despite prompt tuning	Loses demo differentiator	B	Fallback rule: <i>water stain + duration ≥ 3 days → escalate to YELLOW minimum</i> ; eval must still target RED for <code>email-001</code>
API key committed Day 1–4	Disqualification	All	<code>.pre-commit-config.yaml</code> Day 1 optional; daily <code>git log</code> scan from Day 2
July 4 holiday — member unavailable	Slips E2E deadline	All	Core pipeline (M5–M6) done by Jul 3 EOD ; Jul 4 = polish only
Video runs over 5 minutes	Rubric risk	D	Script totals ~4:30; rehearse once; cut Architecture first
Writeup exceeds 2,500 words	Penalty / rushed cuts	D	Track word count daily from Day 2; target 1,800–2,100
Fresh-clone test fails on Day 6	Judge cannot reproduce	A	Run reproducibility test Day 5 , not Day 6 — time to fix

1.11 10. Testing & Validation Playbook

Operational test commands merged from Claude V7.2 §10. Run these at the checkpoints named in §4.

1.11.1 10.1 Continuous eval (daily from Day 2)

```
pytest tests/eval_urgency.py -v
```

Expected output includes:

- Pass/fail per fixture
- Full 3×3 **confusion matrix** (RED / YELLOW / GREEN)
- Overall accuracy (target ≥ 90%)
- **False-RED rate** (actual GREEN/YELLOW, predicted RED) — target < 15%
- **False-GREEN rate** (actual RED, predicted GREEN/YELLOW) — target < 2% (**critical**)

1.11.2 10.2 Hard-case validation (Day 3 — blocking)

```
pytest tests/eval_urgency.py -k "email-001 or hard_case_water_stain" -v
```

Must return **PASSED**. If **FAILED**, fix before other Day 3 work — the demo cannot show what the PRD claims.

1.11.3 10.3 State machine validation (Day 1 — screenshot for Writeup)

```
# tests/test_state_machine.py should cover this; manual shell optional:
from db.engine import SessionLocal
from db.models import DraftRow
from sqlalchemy.exc import IntegrityError

session = SessionLocal()
try:
    session.add(DraftRow(
        id="proof-sent-rejected",
        classified_item_id="classified-proof",
        draft_text="test",
        status="sent",
        approved_by=None,
        approved_at=None,
    ))
    session.commit()
    print("FAIL — constraint did not fire")
except IntegrityError as e:
    print(f"PASS — constraint fired: {e}")
finally:
    session.rollback()
    session.close()
```

Screenshot the PASS output — include in Writeup as HITL evidence.

1.11.4 10.4 DB integrity query (Day 5 — screenshot for Writeup)

After a full overnight batch:

```
SELECT status, COUNT(*) FROM drafts GROUP BY status;
```

Confirm: **zero rows** with `status='sent'` . Member C shares screenshot with D for Writeup embed.

1.11.5 10.5 Git hygiene scan (Day 5 — Member A)

```
git log -p --all | grep -iE "api[_-]?key|AIza|Bearer sk-[AKIA|ghp_" \
| grep -v ".env.example" \
| grep -v "your-gemini-key-here" \
| grep -v "dev-token-placeholder"
```

Zero output = safe to make repo public. If any real key is found: stop pushes, run `git filter-repo` , rotate the compromised key.

1.11.6 10.6 Reproducibility test (Day 5 — Member A)

```
cd /tmp
git clone https://github.com/<org>/nightshift.git nightshift-fresh
cd nightshift-fresh
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # add a real Gemini API key locally for this test only
python db/init_db.py
python main.py run-overnight
pytest tests/ -q
```

Must complete without errors. Member B independently verifies README setup steps match this flow.

1.11.7 10.7 ResponseAgent draft quality (Day 5 — Member C)

Read **5 randomly selected** `ResponseAgent` drafts aloud. Confirm they sound warm and property-specific (personality notes used). Flag any generic/robotic drafts for one more prompt-tuning pass.

1.12 11. Stretch Goals (only after Day 4 checkpoint green)

Goal	Owner	Notes
Real Gmail read-only inbox	A	TDD §2.2 stretch; same <code>RawItem</code> contract
Cloud Run deploy + <code>cloudbuild.yaml</code>	A, D	TDD §2.6 optional
Web UI instead of CLI for Morning Brief	C	Nice for video; CLI sufficient for rubric
Policy Server / code-review agent	D	TDD §2.5 full hybrid — <code>policy/check_no_send.py</code> is required; full review agent is stretch

1.13 12. PDF vs This Plan — Gap Summary

Content merged from `NightShift - Team Execution Plan-Claude.pdf` into this document:

Topic	PDF emphasis	Now in this plan
Win/lose priorities	3-item short list	§1 “Three things that will actually win or lose”
Day 1 state machine	Non-negotiable before agents write	Day 1 C tasks + checkpoint
<code>models.py</code> shared imports	Single source of truth	§3 + directory scaffold
CC-BY-4.0 <code>LICENSE</code>	Day 1, not later	Day 1 A + §8.1
Messy fixtures (~20)	Realistic inbox noise	Day 1 B + MCP contracts
Google Doc Writeup	Collaborative draft	Day 1 D + D11 deliverable
<code>sent</code> status DB test	Screenshot for judges	Day 3 C + §8.1
Submit by 6 PM PT	6-hour buffer	Header + Day 6 schedule
Kaggle team merge	Before deadline	Day 6 + §8.2
Video title + scripted reasoning	Exact on-screen text	§5.4 video table
Docker health-check	<code>docker run</code> proof	Day 4 A + A8
Peer review writeup	2 members vs code	Day 4 D
Clean-clone test	Reproducibility if you win	Day 5 A + §8.1
Role split	B=ingestion, C=triage+response	Reconciliation table §2 (this plan keeps approved split)

Already stronger in this plan than the PDF: integration-owner rotation, mermaid dependency graphs, PRD success-metrics ownership table, risk register, stretch goals, handoff deadlines, OTel/security split for Member D, and **V7.4 operational playbook** (§2.1, §4.0, §6.1–6.3, §10).

1.14 13. Claude V7.2 Markdown vs V7.4 — Gap Summary

Content merged from `NightShift_ExecutionPlan-claudeV7.2.md` in **V7.4** (see §0.11):

Topic	Claude V7.2 emphasis	Now in V7.4
Assumptions & constraints	Kaggle phone verify, team merger, no dataset	§2.1
Tools & credentials per member	Full stack + who needs what	§2.2
All-member Day 1 onboarding	Clone → venv → <code>agents-cli --version</code>	§3.10
Milestones M1-M10	Single tracking table	§4.0
Git PR policy + dual review on schema	A approves; A+C for <code>models</code> / <code>db</code>	§6.1
Escalation + shared resources	<code>#nightshift</code> , filter-repo on key leak	§6.2–6.3
Testing playbook	Git scan, reproducibility bash, SQL screenshot	§10
July 4 holiday buffer	Core E2E by Jul 3	§4.0 note + §9
Video timing budget ~4:30	Cut Architecture first	§8.4
Cover image minimum spec	Dark bg + white text	§8.2
Incognito public repo check	Judge-realistic verify	§8.1
<code>approved_by</code> NOT NULL at approve	Day 1 C checkpoint	Day 1 checkpoint + §8.1

Intentionally not merged: personal names (roster stays **Member A/B/C/D**); Claude V7.2 weekday labels (incorrect for Jul 2026).

1.15 14. Copilot V1.0 vs This Plan — Gap Summary

Content merged from `NightShift – Detailed Execution Plan-V1.0-copilot-.md`:

Topic	Copilot emphasis	Now in this plan
Granular directory tree	Per-agent files, <code>db/</code> , <code>observability/</code> , <code>docker/</code>	§3.4 full tree
<code>models/core.py</code> package	Not flat <code>models.py</code>	§3.1, §3.4
<code>db/engine.py</code> + <code>db/fsm.py</code>	SQLite + FSM validator code	§3.3, §3.4, A1b
<code>.env.example</code> vars	<code>MCP_BASE_URL</code> , <code>DB_URL</code> , <code>OTEL_*</code>	§3.5
<code>pyproject.toml</code> deps	Pinned package list	§3.5 table
<code>memory/data/</code> JSON schemas	tenant map + personality	§3.6
Hard-case fixture example	<code>email-001</code> , <code>tenant-123</code>	§3.7
Eval fixture format	<code>expected_urgency_tier</code> in same JSON shape	§3.7
MCP HTTP paths	<code>/read_inbox</code> , etc. + <code>mcp/auth.py</code>	§3.2, §3.4, §3.7
<code>hitl_state_machine.feature</code>	Second Gherkin file	Day 1 D, B6
<code>test_state_machine.py</code>	Dedicated FSM tests	Day 1 C, B8
<code>ThreadPoolExecutor</code> concurrency	<code>max_workers=5</code>	Day 2 A
Starter code snippets	agents, supervisor, eval harness	§15 appendix
Day 7 buffer	Optional polish day	§4 Day 7
Role split	B=ingestion, C=triage+response+DB	Reconciliation table §2

Already stronger in this plan than Copilot: member definition-of-done, integration checkpoints, rubric traceability (§7), submission checklist (§8), risk register, video/writeup scripts, and explicit handoff deadlines.

1.16 15. Reference Implementation Patterns (starter code)

Use these as **Day 1 drop-in targets** — Member A implements inside the **ADK 2.0 graph**; others import. The patterns below show routing and contracts; the actual entrypoint must use ADK sub-agents per TDD §2.1 and capstone Key Concept #1.

Important: A plain `ThreadPoolExecutor` calling `run_triage()` / `run_response()` functions is **not** sufficient for submission — it is only the scheduling pattern **inside** the ADK `SupervisorNode`.

1.16.1 15.1 ADK supervisor skeleton (`main.py` — target shape)

```
# google-adk 2.0 - three sub-agents + non-LLM SupervisorNode
# CONCURRENCY = 5 schedules items inside supervisor (TDD §2.2)

def process_item(raw_item: RawItem, adk_session, db_session) → None:
    try:
        classified = triage_agent.run(raw_item, session=adk_session) # ADK TriageAgent
        draft = response_agent.run(classified, session=adk_session, db=db_session) # always staged
    except Exception:
        mark_triage_failed(raw_item.id, db_session) # do not halt batch
```

1.16.2 15.2 IngestionAgent MCP client pattern (inside ADK IngestionAgent)

```
def _call_mcp(path: str, params: dict | None = None) → list[RawItem]:
    headers = {"Authorization": f"Bearer {MCP_TOKEN}"}
    resp = httpx.get(f"{MCP_BASE_URL}{path}", headers=headers, params=params or {})
    return [RawItem(**item) for item in resp.json()]

def run_ingestion() → list[RawItem]:
    return _call_mcp("/read_inbox") + _call_mcp("/read_hoa_portal") + _call_mcp("/read_invoices_folder")
```

1.16.3 15.3 Eval confusion matrix (`tests/eval_urgency.py`)

Fixtures load from `tests/fixtures/inbox_test.json` with `(RawItem, expected_urgency_tier)` pairs. Report full 3×3 matrix; assert `accuracy ≥ 0.9` and monitor false-GREEN cell specifically.

1.16.4 15.4 Architecture diagrams for README

ASCII graph (Copilot §7.1) for terminal-friendly README; Mermaid graph (§1 above) for Writeup and `docs/architecture.png`.

1.17 16. Quick Reference Commands (target state)

```
# Setup (Member A maintains)
git clone <repo-url> && cd nightshift
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # add GEMINI_API_KEY locally - never commit

# Run mock MCP server (Member A) - default http://localhost:8000
uvicorn mcp.server:app --reload --port 8000

# Overnight batch (ADK graph)
agents-cli run .
# or: python main.py

# Policy check (SDD - Day 4)
python policy/check_no_send.py

# Init DB tables (Member C - first run)
python -c "from db.engine import engine, Base; from db.models import DraftRow;
          Base.metadata.create_all(engine)"

# Morning Brief (manager view)
python main.py morning-brief

# Eval suite + state machine
agents-cli test
# or: pytest tests/eval_urgency.py tests/test_state_machine.py -v

# Approve item (Member C)
python main.py approve --draft-id <id> --manager "Jane Doe"
```

End of Execution Plan V7.4. For product requirements see [NightShift_PRD-V7.3.md](#). For technical design see [NightShift_TDD-V7.3.md](#).